Hopfield Networks under Biological Constraints

NX-465 Mini-Project MP1

Maria Cherchouri
Aicha Masmoudi

April 25, 2026

1 Description

In this project, we look at how Hopfield networks can store and recall patterns, and how adding more realistic features, i.e. biological constraints, changes their performance. In the basic Hopfield model, we store P balanced bipolar patterns by setting $w_{ij} = \frac{1}{N}\sum_{\mu=1}^{P} p_i^\mu p_j^\mu$, then update each neuron by iterating a sign-threshold update.

We first investigate the storage capacity of the standard Hopfield network e.g. by measuring how many bit-flips each pattern can survive (the basin of attraction) and how many patterns the network can store (its capacity) as we vary P and N. Next, we study the storage capacity in the network but with diluted connections, so each neuron keeps only K/N of its connections, and see how this pruning changes basins of attraction and capacity. Finally, we explore biologically realistic dynamics by introducing stochastic firing: $P(\sigma_i = 1 \mid h_i) = \frac{1}{2}\big[1 + \tanh(\beta h_i)\big]$ along with an absolute refractory rule (no two spikes in a row) to study how noise and refractoriness influence memory recall.

2 Ex 0. Getting Started: standard Hopfield network

In Exercise 0, we generate P balanced random patterns $p^\mu \in \{\pm 1\}^N$ and build the Hebbian weight matrix. We then implement both the full synchronous update $S_i(t+1) = \mathrm{sgn}(\sum_{j=1}^{N} w_{ij} S_j(t))$ with $h_i(t) = \sum_{j=1}^{N} w_{ij} S_j(t)$ and the faster overlap-based update via $m^\mu(t) = \frac{1}{N}\sum_{i=1}^{N} p_i^\mu S_i(t)$ and $h_i(t) = \sum_{\mu=1}^{P} m^\mu(t)\, p_i^\mu$.

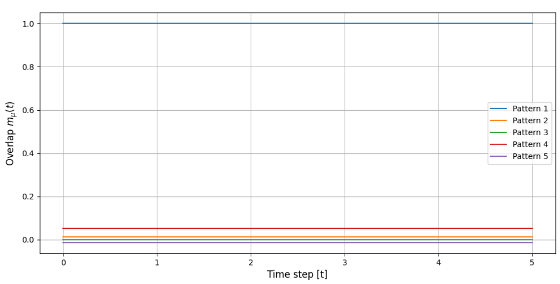
Ex 0.3: What is the gain in the computational cost of a single faster overlap-based update step? The standard computation of the input to each neuron involves a full matrix-vector multiplication, where the weight matrix w_{ij} has size $N \times N$. Thus, computing $h_i(t)$ for all $i = 1, \ldots, N$ requires $\mathcal{O}(N^2)$ operations. In opposite, the faster method requires $\mathcal{O}(PN)$ operations to compute all $m^\mu(t)$, and $\mathcal{O}(PN)$ operations to compute all $h_i(t)$, so the total cost is $\mathcal{O}(PN)$. So we can say that when $P \ll N$, the overlap-based method is significantly faster than the full matrix multiplication, reducing the computational cost of a state update step from $\mathcal{O}(N^2)$ to $\mathcal{O}(PN)$.

3 Ex 1. Storage capacity in the standard Hopfield network

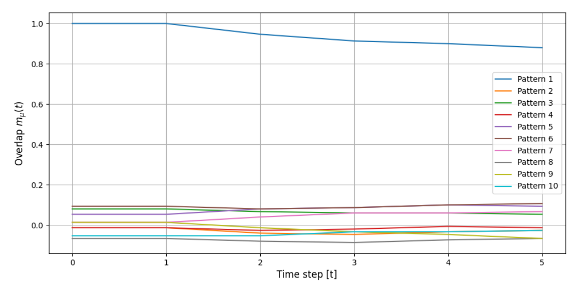
In Exercise 1, we first generate P balanced random patterns and use the fast-update. In 1.1–1.4, we initialize the network on a stored pattern (with optional bit-flips) and plot the time course of overlaps $m^\mu(t)$. In 1.5 ("basin of attraction"), we measure the fraction of patterns retrieved perfectly as a function of fraction flipped, for various P. In 1.6 ("capacity"), we identify $P_{\max}$, the maximum number of patterns the network can store without errors (over 5 repeats) for $N \in \{100, \ldots, 1500\}$ to estimate the capacity $\alpha = P_{\max}/N$.

Ex 1.1: Why does the overlap $m^1(t)$ remain constant across time? It remains constant across time because the initial state of the network is set to the first stored pattern: $S(0) = p^1$. When the number of stored patterns P is small compared to the number of neurons N, the stored patterns become fixed-point attractors of the Hopfield dynamics. Since the initial state already matches one of the attractors, the network remains in that state for all next steps: $S(t) = p^1$ for all t. As a result, the overlap with the first pattern remains: $m^1(t) = \frac{1}{N}\sum_{i=1}^{N} p_i^1 S_i(t) = \frac{1}{N}\sum_{i=1}^{N} p_i^1 p_i^1 = 1$, and the overlaps with the other patterns, remain approximately constant and close to zero since the stored patterns are random and uncorrelated with one another. This behavior is clearly illustrated in Figure 1a.

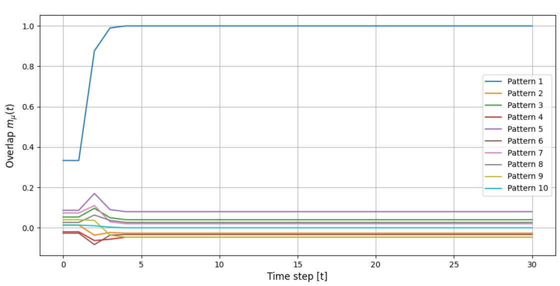
Ex 1.2: Why does the overlap $m^1(t)$ drop below 1? What does the network converge to instead of the first pattern? The number of stored patterns is very large: $P = 200$ with $N = 300$. This corresponds to a memory load of: $\alpha = P/N \ = \frac{200}{300} \approx 0.67$ which is significantly higher than the theoretical capacity limit of the Hopfield model, $\alpha \approx 0.138$, so the network exceeds its storage capacity. As a result, interference between the stored patterns grows, and the dynamics no longer preserve each pattern as a stable fixed point. This prevents the network from staying in or returning to the correct attractor p^1. The overlap $m^1(t)$ drops below 1, and the network state begins to drift away from the initial pattern. This is confirmed in Fig 1b, $m^1(t)$ decreases rapidly, and no clear convergence to any single pattern is observed.
Ex 1.3: Why does the overlap $m^1(t)$ increase to 1 across time? The network is initialized with a corrupted version of the first stored pattern p^1, where 200 out of the 600 bits have been flipped. The initial overlap is therefore: $m^1(0) = \frac{1}{N}\sum_{i=1}^{N} p_i^1 S_i(0) = \frac{600-2\cdot 200}{600} = \frac{200}{600} = 0.33$. Despite this significant noise, the network gradually converges back to the original pattern. This is because the Hopfield model is designed to perform pattern completion and because the memory load is low ($\alpha = P/N = 20/600 = 0.033 \ll 0.138$), so each stored pattern remains a strong attractor. As observed in Fig. 1c, $m^1(t)$ increases monotonically and reaches 1, indicating perfect retrieval.
Ex 1.4: What happens when the first pattern is not correctly retrieved? The network is initialized with a version of the first stored pattern p^1 in which 300 out of 600 bits are flipped. This gives an initial overlap of: $m^1(0) = \frac{600-2\cdot 300}{600} = 0$. As shown in Figure 1d, the network fails to retrieve the correct pattern. The overlap $m^1(t)$ fluctuates slightly but never converges to 1, indicating that the state has instead fallen into a spurious attractor. This highlights the limit of the network's basin of attraction, if the initial distortion is too large, the dynamics cannot recover the original memory.



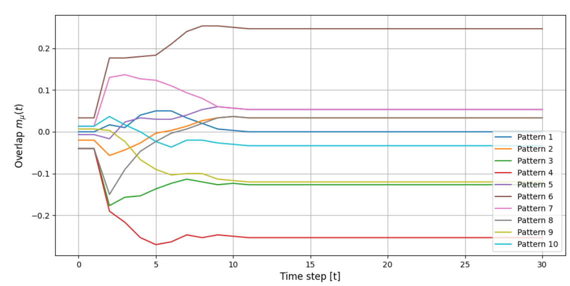
(a) $N = 300$, $P = 5$, $T = 5$



(b) $N = 300$, $P = 200$, $T = 5$



(c) $N = 600$, $P = 20$, $T = 30$, $n_{\text{flip}} = 200$

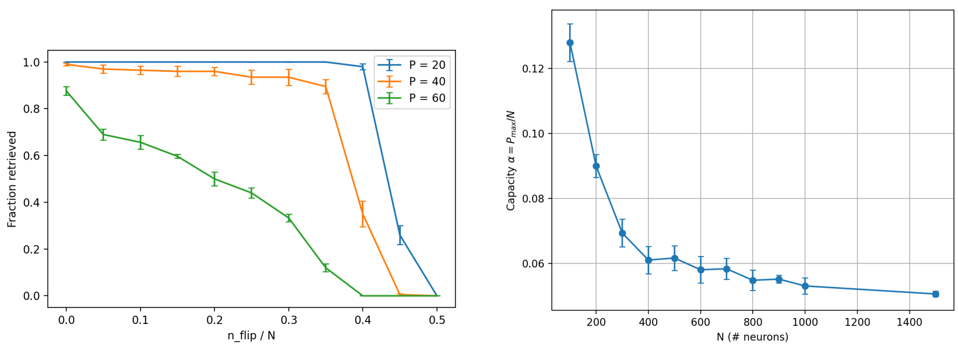


(d) $N = 600$, $P = 20$, $T = 30$, $n_{\text{flip}} = 300$

Figure 1: Time courses of the overlap $m_\mu(t)$ for various network sizes, pattern counts, and flip levels.

Ex 1.5: Describe and explain the trends for the three curves (one for every value of P). The basin of attraction represents the fraction of patterns a network successfully retrieves. Figure 2a shows that with no flips ($n_{\text{flip}}/N = 0$), all patterns are retrieved perfectly (1.0) for $P = 20$ and $P = 40$, and most of them are (≈ 0.9) for $P = 60$. As flips increase, the lightly loaded network ($P = 20$) remains at 1.0 until $\sim 40\%$ flips then collapses to zero by $\sim 50\%$. The medium load ($P = 40$) holds until $\sim 20\%$ and drops to zero near $\sim 45\%$, while the heavy load ($P = 60$) declines immediately, reaching zero by $\sim 40\%$. Small standard-error bars indicate low trial-to-trial variability. Thus, these results illustrate that (1) a very high level of corruption of inputs (i.e. more flips) dramatically lowers the basin of attraction, and (2) a higher number of patterns P increases cross-talk and shrinks basin of attraction,

making the network far less robust to corrupted (flipped) inputs.
Ex 1.6: Investigate how capacity changes as N increases from 100 to 1500 Figure 2b shows that the normalized capacity $\alpha = P_{\max}/N$ falls from about 0.125 at $N = 100$ to ~ 0.045 by $N = 1500$. Error bars over five trials are relatively small, slightly larger at small N, indicating low trial-to-trial variability. It may seem counterintuitive that α decreases as N grows, since one might expect larger networks to store proportionally more patterns. However, our strict one-step update requirement (demanding perfect stability in a single update) and the finite number of repeats both tend to underestimate the true capacity. In contrast, the classic result $\alpha_c \approx 0.14$ allows a small error rate and asynchronous relaxation over many steps, so it remains higher than our measured values.
Ex 1.7 (Bonus): What would now be the capacity α in the limit of infinite network ($N \to \infty$)? In the strict zero-error regime ($P_{\text{error}} = 0$), even if the theoretical error probability under the Gaussian approximation is very small, it still counts as a failure. Recall the definition of the complementary error function $\text{erfc}(x) = 1 - \text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_x^\infty e^{-t^2} dt$. The one-step error probability can then be written as $P_{\text{error}}(\alpha) \approx \frac{1}{2} \text{erfc}\left(\frac{1}{\sqrt{2\alpha q}}\right)$. Setting $P_{\text{error}} = 0.01$ gives $\alpha \approx 0.15$, while $P_{\text{error}} \to 0$ forces the erfc argument, and hence α, to zero. Thus in the $N \to \infty$ limit, demanding perfect one-step stability for all neurons yields $\alpha = \lim_{N\to\infty} \frac{P_{\max}}{N} = 0$.



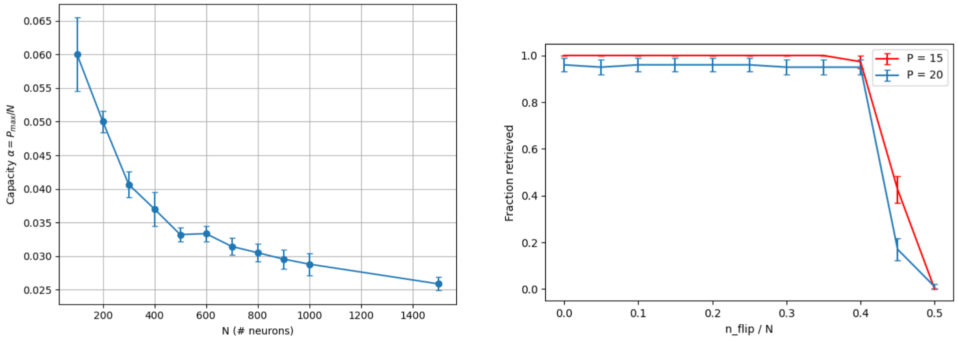
(a) Fraction retrieved vs. corruption level for $P = 20, 40, 60$.

(b) Normalized capacity $\alpha = P_{\max}/N$ vs. network size N.

Figure 2: (a) Basin-of-attraction curves showing retrieval fraction as a function of flip fraction for various P. (b) Storage capacity α as a function of network size N.

4 Ex 2. Storage capacity in the Hopfield network with diluted connections

In Exercise 2, we introduce random dilution ($K = 0.5N$) into the Hopfield network by masking its Hebbian weights, then repeat the basin-of-attraction and capacity experiments from Exercise 1 to quantify how pruning connections shrinks attractor basins and lowers storage capacity.
Ex 2.2: Redo question 1.6 but with the diluted network. Compare the results and make a comment. Figure 3a shows that dilution ($K = 0.5N$) cuts the normalized capacity $\alpha = P_{\max}/N$ roughly in half—from about 0.06 at $N = 100$ to 0.025 at $N = 1500$, compared to the fully connected case (Figure 2b), which drops only from 0.125 to 0.05. In absolute terms, a diluted vs. fully-connected network stores 6 vs. 12 patterns at $N = 100$ and 37 vs. 75 at $N = 1500$. Moderate error bars indicate similar small trial-to-trial variability in both networks. Thus, dilution roughly halves the storage efficiency at every network size. In other words, pruning half the synapses drastically lowers (even halves) storage capacity by amplifying cross-talk noise.
Ex 2.3: Redo question 1.5 but with the dilutd network. Compare the results and make a comment. Figure 3b shows that with dilution ($K = 0.5N$), both $P = 15$ and $P = 20$ maintain near-perfect recall up to $n_{\text{flip}}/N \approx 0.4$ but fall to zero at $n_{\text{flip}}/N = 0.5$. Unlike the fully connected case (Figure 2a), where $P = 20$ stays at 1.0 until ~ 0.4, the diluted network's $P = 20$ curve peaks at only ~ 0.95 even at zero flips, indicating weaker fixed-point stability. Thus, pruning half the synapses lowers the maximum achievable overlap and significantly weakens attractor basins under noise. It highlights how full connectivity maximizes fixed-point fidelity in Hopfield networks.



(a) Capacity $\alpha = P_{\max}/N$ vs. N (diluted, $K = 0.5N$).

(b) Retrieval fraction vs. flip fraction for $P = 15, 20$ (diluted, $K = 0.5N$).

Figure 3: Diluted Hopfield network ($K = 0.5N$) results: (a) normalized storage capacity, (b) basin-of-attraction curves.

Ex 2.4: Assuming $K = 0.5N$ and $P \ll N$, show mathematically that the patterns remain fixed points of the diluted network dynamics—i.e., if $S(t) = p^\mu$, then $S(t+1) = S(t)$—in the limit $N \to \infty$. Assume $S_i(t) = p_i^\nu$. The one-step update in the diluted network is $S_i(t+1) = \operatorname{sgn}(h_i), \quad h_i = \sum_j C_{ij} w_{ij} p_j^\nu$, with $w_{ij} = \frac{1}{N}\sum_\mu p_i^\mu p_j^\mu$. Hence, by using $p_j^\nu p_j^\nu = 1$,

$$h_i = \sum_{j=1}^{N} C_{ij}\Big(\tfrac{1}{N}\sum_{\mu=1}^{P} p_i^\mu p_j^\mu\Big)p_j^\nu = \frac{1}{N}\sum_{\mu=1}^{P} p_i^\mu \sum_{j=1}^{N} C_{ij}\, p_j^\mu\, p_j^\nu = \underbrace{\frac{1}{N}\, p_i^\nu \sum_j C_{ij}}_{\text{signal}} + \underbrace{\frac{1}{N}\sum_{\mu\neq\nu} p_i^\mu \sum_j C_{ij} p_j^\mu p_j^\nu}_{\text{crosstalk}} .$$

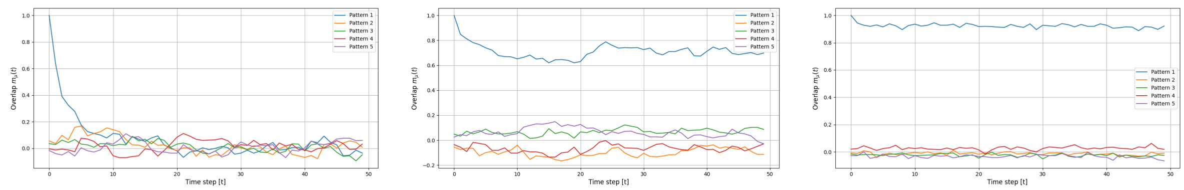
Since each neuron has $K = 0.5N$ inputs, $\frac{1}{N}\sum_j C_{ij} = \frac{K}{N} = \frac{1}{2}$, and the "signal" term becomes $\frac{p_i^\nu}{N}\sum_j C_{ij} = \frac{1}{2}p_i^\nu \implies \operatorname{sgn}(\frac{1}{2}p_i^\nu) = p_i^\nu$. The "crosstalk" term is a sum of $P-1$ independent zero-mean terms of size $1/N$, hence $\frac{1}{N}\sum_{\mu\neq\nu}\sum_j C_{ij}p_j^\mu p_j^\nu \to 0$ as $N \to \infty$ by the CLT (Central-Limit Theorem). Thus $h_i \approx \frac{1}{2}p_i^\nu, \quad S_i(t+1) = \operatorname{sgn}(h_i) = p_i^\nu$, so p^ν is a fixed point.

5 Storage capacity in the Hopfield network with more realistic firing output

In Exercise 3 we extend the Hopfield model to *stochastic firing* using the faster overlap-based update and compare retrieval dynamics for $\beta \in \{1.5, 2.5, 3.5\}$ (3.1) and estimate capacity as a function of β (3.2–3.3). We then add an *absolute refractory period* ($\sigma_i = 1 \Rightarrow \sigma_i(t+1) = 0$), study its impact on forgetting and basin rescue strategies at high β (3.4–3.6), and finally chart capacity vs. β with refractory scaling (3.7).

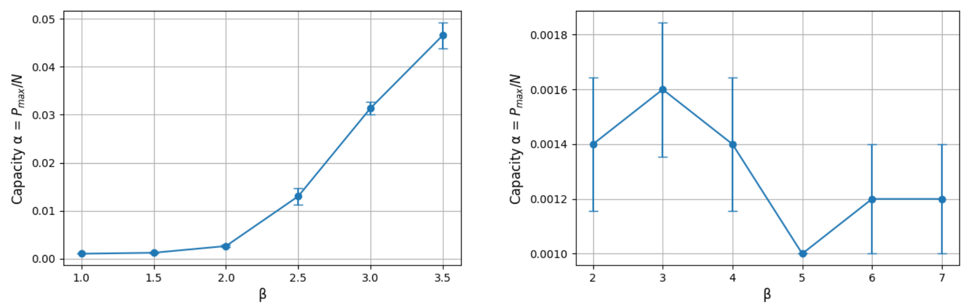
Ex 3.1: Is the first pattern stored properly? What is the resulting overlap with the first pattern? Compare the results for $\beta = 1.5$, $\beta = 2.5$, and $\beta = 3.5$, and provide a brief comment. We evaluated whether the first pattern is properly stored in a stochastic Hopfield network by analyzing the average overlap between the network state and the stored patterns over the last 10 time steps, under different values of β. In Fig.4, we see that for $\beta = 1.5$, the first pattern is not retrieved, as the average overlap is 0.009 and no pattern clearly dominates, indicating that the high level of noise prevents the network from stabilizing on a memory. For $\beta = 2.5$, the first pattern is retrieved with a mean overlap of 0.709, and for $\beta = 3.5$, the retrieval is even stronger, with a high and stable mean overlap of 0.914, demonstrating that a moderate reduction in stochasticity allows the network to converge toward the correct memory despite some fluctuations. This comparison highlights the importance of *beta* in controlling the trade-off between noise and stability. Higher β values reduce the randomness in neuron updates, allowing the network to more effectively retrieve stored patterns.

Ex 3.2: Describe the trend of how the capacity changes with respect to β. In Fig.5a we observe, as β increases, the stochastic Hopfield network transitions from a noise dominated regime to a memory dominanted regime. Around $\beta = 2.0$, a clear phase transition occurs, enabling the retrieval of stored patterns. At higher β values, capacity increases rapidly, showing that neuronal reliability is crucial for effective associative memory storage in stochastic networks.



(a) $\beta = 1.5$ (b) $\beta = 2.5$ (c) $\beta = 3.5$

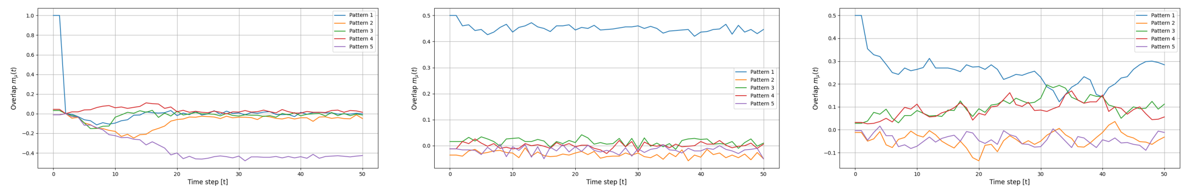
Figure 4: Time courses of the overlap $m_\mu(t)$ for stochastic-firing Hopfield networks with $N = 1000$, $P = 5$, and various β during $T = 50$ steps.



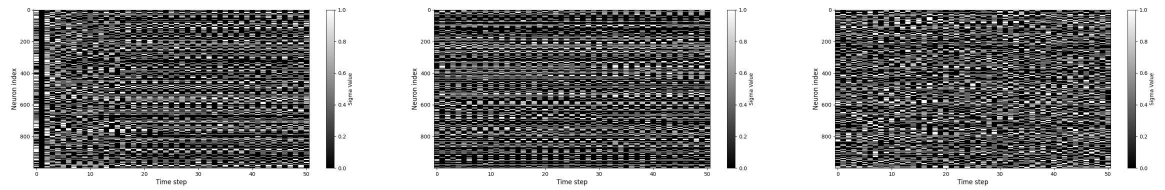
(a) without refractory period (b) with refractory period.

Figure 5: Normalized Storage Capacity as a function of β.

Ex 3.3 (Bonus): Show mathematically that there is a phase transition at $\beta = 2.0$ in the limit $N \to \infty$. Considering the stochastic Hopfield network with binary outputs $\sigma_i \in \{0, 1\}$, updated according to the probability describe in the Introduction, where the input to each neuron is: $h_i = \sum_j w_{ij}\sigma_j$. Assume that P balanced random patterns $p^\mu \in \{-1, +1\}^N$ are stored using the Hebbian rule. We analyze the expected behavior of the network initialized near the first stored pattern p^1. Define the overlap with pattern μ at time t as: $m^\mu(t) = \frac{2}{N}\sum_{i=1}^{N} p_i^\mu \sigma_i(t)$. Assuming other overlaps $m^{\mu \neq 1} \approx 0$, the input becomes: $h_i \approx \frac{1}{2}m(t)p_i^1$.
The expected value of $\sigma_i(t+1)$ becomes: $E[\sigma_i(t+1)] = \frac{1}{2}\left(1 + \tanh\left(\frac{1}{2}\beta m(t)p_i^1\right)\right)$.
Then the expected overlap at the next time step is: $E[m(t+1)] = \frac{2}{N}\sum_{i=1}^{N} p_i^1 \cdot E[\sigma_i(t+1)]$.
Using symmetry of $p_i^1 \in \{-1, +1\}$, we split the sum into positive and negative cases: $E[m(t+1)] = \tanh\left(\frac{1}{2}\beta m(t)\right)$. Thus, the recurrence equation for the expected overlap is: $m_{t+1} = \tanh\left(\frac{1}{2}\beta m_t\right)$.
We analyze the fixed point $m = \tanh\left(\frac{1}{2}\beta m\right)$. This always has a solution at $m = 0$. To determine its stability, we linearize around $m = 0$: $\tanh\left(\frac{1}{2}\beta m\right) \approx \frac{1}{2}\beta m$, so $m_{t+1} \approx \lambda m_t, \quad$ with $\lambda = \frac{1}{2}\beta$. So to conclude: if $\lambda < 1$ (i.e., $\beta < 2$), then $m(t) \to 0$ is stable: the system forgets the pattern. And if $\lambda > 1$ (i.e., $\beta > 2$), then $m(t)$ grows and converges to a nonzero fixed point: the system retrieves the pattern.
Therefore, the critical point of phase transition occurs at: $\lambda = 1$, which means $\beta = 2$.
Ex 3.4: Is the first pattern stored properly? If not, what is the reason? We introduced a refractory period. As shown in Fig. 6a, the overlap $m^1(t)$ drops immediately from 1 to near zero, and does not recover. This indicates that the first pattern is not retrieved. Figure 6d shows the network state over time. At $t = 0$, most neurons are active (since $\sigma(0)$ corresponds to p^1), but at $t = 1$, activity drops drastically due to the refractory condition. After this, the network enters a noisy regime without clear structure. The refractory period forces a large number of neurons to turn off after the initial step, effectively erasing the memory trace of the stored pattern. This prevents the network from recovering the correct attractor, and retrieval fails.
Ex 3.5: Rescue the network above from 'forgetting' the pattern by initializing the network smartly. To mitigate the forgetting observed we used an initialization strategy: instead of activating all neurons from pattern p^1, we randomly selected some active bits to reduce simultaneous firing. As shown in Figure 6b, the overlap $m^1(t)$ starts around 0.5 and remains stable throughout the simulation. While it does not reach 0.5 exactly, it consistently stays close, and remains significantly higher than the overlaps with any other pattern. This indicates that the memory of the first pattern is at least



(a) $\beta = 7.0, init = normal$ (b) $\beta = 7.0, init = smart$ (c) $\beta = 5.0, init = smart$



(d) $\beta = 7.0, init = normal$ (e) $\beta = 7.0, init = smart$ (f) $\beta = 5.0, init = smart$

Figure 6: Time courses of the overlap $m_\mu(t)$ for stochastic-firing Hopfield networks with $N = 1000$, $P = 5$, and various β during $T = 50$ steps.

partially retained. The network avoids the collapse seen in Ex 3.4, and the heatmap confirms a more structured firing pattern over time. Smart initialization effectively mitigates the destructive effect of the refractory rule. Even though the network does not achieve perfect retrieval, the dominant and stable overlap with p^1 suggests a successful rescue of the memory trace.

Ex 3.6: What do you observe this time? Compare the dynamics of the network states and overlaps with what you observed in the previous question, and comment on the results.
With $\beta = 5.0$, Fig.6c the network begins with an initial overlap $m^1(0) \approx 0.5$, but the value quickly drops and fluctuates between 0.2 and 0.3 throughout the simulation. In contrast to the stable retrieval observed at $\beta = 7.0$, here the target pattern no longer clearly dominates. Competing overlaps $m^\mu(t)$ slightly increase, and the network state becomes noisier and less structured, as shown in the heatmap. Even if smart initialization helps avoid immediate forgetting, the added stochasticity from a lower β prevents full convergence to the correct memory. This highlights the trade-off between noise and memory stability: even with a good initialization, a sufficiently high β remains essential for successful recall under refractory dynamics.

Ex 3.7 (Bonus): Compare the results with question 3.2 and comment on the results.
In Fig.5b we observe, the capacity peaks at $\beta = 3$, with a maximum around $\alpha \approx 0.0016$. For lower values ($\beta = 2$), the network becomes too noisy to reliably retrieve patterns. For higher values ($\beta \geq 5$), capacity drops significantly. This counterintuitive behavior is due to the interaction between deterministic updates and the refractory rule. As β increases, neuron firing becomes more synchronized. This causes large groups of neurons to spike simultaneously and then be silenced in the next step due to refractoriness. The resulting oscillatory or degraded dynamics reduce the number of patterns that can be reliably stored and retrieved. Under refractory dynamics, there is an optimal range of β that balances reliability and diversity in neuron activity. Very high β values, while beneficial in standard networks, hurt performance when refractoriness is present. These results align with the theory that scaling up weights does not always translate into higher capacity in biologically realistic models.

Conclusion

This project explored how biological constraints influence memory in Hopfield networks. We showed that reducing connectivity lowers capacity and stability but doesn't eliminate memory storage. Stochastic firing introduces a phase transition around $\beta = 2$, marking the shift from noise to reliable recall. With a refractory period, very high β causes synchronous firing and memory loss, while moderate β offers better balance. Good initialization can prevent collapse and support partial retrieval. Together, these results show that adding biological realism doesn't destroy memory, but reshapes the dynamics. Effective memory storage under realistic conditions requires tuning the trade-off between noise, structure, and stability.